

TrustIQ — COMPLETE 3-MODULE BACKEND PATHWAY

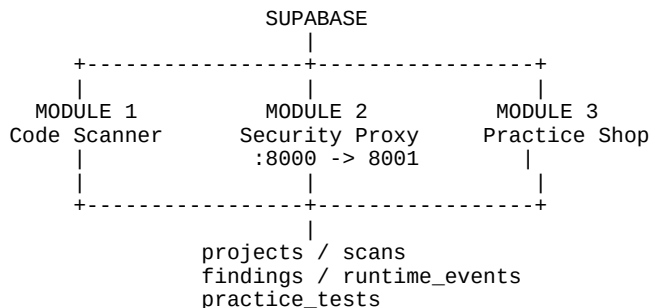
After Supabase is connected: code, integration order, testing, and end-to-end operation

Scope: This is the complete pathway for your existing TrustIQ architecture: Module 1 Code Inspector, Module 2 Security Guard/Zero-Trust Proxy, and Module 3 Practice Shop. It starts **after the Supabase connection is established** and explains what must be added, where it belongs, and in what order to test it.

Important honesty: No document can guarantee a successful run without executing your exact current files. Your existing Module 1/2/3 code has project-specific function names and logic. Therefore this guide uses integration code that can be inserted around that code rather than pretending a generic replacement is your exact implementation. Test each stage before moving on.

Security: Use only fake credentials in the intentionally vulnerable Practice Shop/demo repository. Keep the Supabase backend secret in .env and never commit it.

1. Final architecture



Ports: Module 2 proxy = 8000. Module 3 Practice Shop = 8001. Module 1 is a scanner and is not a replacement for either server. Module 2 forwards permitted traffic to Module 3.

2. Existing Supabase tables and ownership

Table	Primary role	Module
projects	Project/repository identity	Module 1 + shared
scans	Scan execution and summary	Module 1
secret_rules	Hardcoded secret/PII detection rules	Module 1
sensitive_files	Dangerous filename rules	Module 1
dependency_rules	Package/CVE rules	Module 1
findings	Detected Module 1 findings; also schema supports module field	Module 1/shared
security_rules	Runtime request/response security rules	Module 2
runtime_events	Runtime security audit events	Module 2
practice_scenarios	Expected Practice Shop security tests	Module 3/testing
practice_tests	Actual scenario test results	Module 3/testing

3. Step 1 — Install backend dependencies

```
pip install supabase python-dotenv flask requests
```

If your existing requirements.txt already contains some of these, do not duplicate them unnecessarily. Freeze only after the integration works.

4. Step 2 — Environment configuration

```
# .env
```

```
SUPABASE_URL=https://YOUR_PROJECT_REF.supabase.co
SUPABASE_SECRET_KEY=YOUR_BACKEND_SECRET_KEY
```

```
# Later hardening for Module 2:
JWT_SECRET=YOUR_DEMO_JWT_SECRET
TARGET_URL=http://127.0.0.1:8001
PROXY_PORT=8000
.env
__pycache__/
*.pyc
.venv/
venv/
security_log.json
```

Do not put real credentials into the demo vulnerability files. The .env above is the private backend configuration. The vulnerable demo may contain a separate fake .env file inside the demo repository only.

5. Step 3 — Common Supabase client

```
# supabase_client.py
import os
from dotenv import load_dotenv
from supabase import create_client, Client

load_dotenv()

SUPABASE_URL = os.getenv("SUPABASE_URL")
SUPABASE_SECRET_KEY = os.getenv("SUPABASE_SECRET_KEY")

if not SUPABASE_URL:
    raise RuntimeError("SUPABASE_URL is missing")

if not SUPABASE_SECRET_KEY:
    raise RuntimeError("SUPABASE_SECRET_KEY is missing")

supabase: Client = create_client(
    SUPABASE_URL,
    SUPABASE_SECRET_KEY
)
```

6. Step 4 — Connection test

```
# test_supabase.py
from supabase_client import supabase

for table in ["projects", "secret_rules", "sensitive_files",
              "dependency_rules", "security_rules",
              "practice_scenarios"]:
    result = supabase.table(table).select("*").limit(1).execute()
    print(table, "OK", len(result.data))
```

Do not proceed until this test works.

MODULE 1 — CODE INSPECTOR

7. Step 5 — Module 1 database rule loader

```
# module1_db.py
from supabase_client import supabase

def get_secret_rules():
    return (supabase.table("secret_rules")
            .select("*").eq("enabled", True).execute().data)

def get_sensitive_files():
    return (supabase.table("sensitive_files")
            .select("*").eq("enabled", True).execute().data)

def get_dependency_rules():
    return (supabase.table("dependency_rules")
            .select("*").eq("enabled", True).execute().data)
```

8. Step 6 — Start a scan

```
# module1_db.py (continued)
def create_project(name, repository, owner_id="demo"):
    return (supabase.table("projects")
            .insert({
                "name": name,
                "repository": repository,
                "owner_id": owner_id
            }).execute().data[0])

def start_scan(project_id):
    return (supabase.table("scans")
            .insert({
                "project_id": project_id,
                "status": "running",
                "files_scanned": 0,
                "findings_count": 0,
                "critical_count": 0,
                "high_count": 0,
                "medium_count": 0,
                "low_count": 0
            }).execute().data[0]["id"])
```

9. Step 7 — Preserve and call the existing Module 1 scanner

Do NOT replace your existing scanner. It already contains the secret patterns, sensitive-code checks, PII checks, entropy checking, and dependency scanning you developed. Wrap it with an integration layer. Your exact existing scanner function name must be used here.

```
# module1_integration.py – integration skeleton
from module1_db import (
    get_secret_rules,
    get_sensitive_files,
    get_dependency_rules,
    start_scan,
)
from supabase_client import supabase

def save_finding(scan_id, finding):
    return (supabase.table("findings").insert({
        "scan_id": scan_id,
        "rule_code": finding["rule_code"],
        "module": "MODULE_1",
        "file_name": finding["file"],
        "line_number": finding.get("line"),
        "severity": finding["severity"],
        "message": finding["issue"],
        "recommendation": finding["fix"],
        "status": "OPEN"
    }).execute().data)

# Replace ONLY this call with the function from your existing scanner:
def run_module1(project_id, demo_path):
    rules = {
        "secret_rules": get_secret_rules(),
        "sensitive_files": get_sensitive_files(),
        "dependency_rules": get_dependency_rules(),
    }

    scan_id = start_scan(project_id)

    # findings = existing_module1_scan(demo_path, rules)
```

```

# files_scanned = ...
#
# For each finding returned by your existing scanner:
# save_finding(scan_id, finding)
#
# Then update scans with completed_at and counts.

return scan_id

```

10. Step 8 — Finish Module 1 scan

```

from datetime import datetime, timezone
from supabase_client import supabase

def finish_scan(scan_id, findings, files_scanned):
    counts = {"CRITICAL":0, "HIGH":0, "MEDIUM":0, "LOW":0}

    for f in findings:
        sev = f.get("severity", "").upper()
        if sev in counts:
            counts[sev] += 1

    return (supabase.table("scans").update({
        "completed_at": datetime.now(timezone.utc).isoformat(),
        "files_scanned": files_scanned,
        "findings_count": len(findings),
        "critical_count": counts["CRITICAL"],
        "high_count": counts["HIGH"],
        "medium_count": counts["MEDIUM"],
        "low_count": counts["LOW"],
        "status": "completed"
    }).eq("id", scan_id).execute().data)

```

11. Step 9 — Module 1 demo repository

```

TrustIQ_Module1_Demo/
■■■ vulnerable_secrets.py
■■■ vulnerable_code.py
■■■ .env
■■■ credentials.json
■■■ service-account.json
■■■ private.key
■■■ requirements.txt

```

The demo should cover the rules in `secret_rules`, `sensitive_files`, and `dependency_rules`. Use fake/test values only. Do not use real API keys, passwords, cloud credentials, or personal data.

MODULE 2 — SECURITY GUARD / ZERO-TRUST PROXY

12. Step 10 — Move Module 2 configuration to .env

Your existing Module 2 currently has hardcoded JWT_SECRET and TARGET_URL. Move configuration to environment variables without changing the security pipeline.

```
# at the top of existing module2_proxy.py
import os
from dotenv import load_dotenv

load_dotenv()

TARGET_URL = os.getenv("TARGET_URL", "http://127.0.0.1:8001")
JWT_SECRET = os.getenv("JWT_SECRET")

if not JWT_SECRET:
    raise RuntimeError("JWT_SECRET is missing")
```

13. Step 11 — Common runtime event writer

```
# runtime_db.py
from supabase_client import supabase

def save_runtime_event(
    rule_code,
    event_type,
    ip_address=None,
    http_method=None,
    path=None,
    details=None,
    action=None,
    response_status=None,
    project_id=None,
):
    payload = {
        "rule_code": rule_code,
        "event_type": event_type,
        "ip_address": ip_address,
        "http_method": http_method,
        "path": path,
        "details": details,
        "action": action,
        "response_status": response_status,
        "project_id": project_id,
    }

    return (supabase.table("runtime_events")
            .insert(payload).execute().data)
```

14. Step 12 — Load Module 2 rules from Supabase

```
# runtime_db.py (continued)
def get_runtime_rules():
    return (supabase.table("security_rules")
            .select("*")
            .eq("enabled", True)
            .execute().data)
```

This allows the proxy to use the existing security_rules table. The existing detection pipeline remains: IP block/rate limit → request inspection → auth/authz/IDOR/price checks → forward → response DLP → audit.

15. Step 13 — Insert runtime event at each enforcement point

```
# Example inside existing Module 2 block logic:
save_runtime_event(
    rule_code="SQLI-001",
    event_type="BLOCKED_REQUEST",
    ip_address=request.remote_addr,
    http_method=request.method,
    path=request.path,
    details="SQL injection pattern detected",
    action="BLOCK",
    response_status=403,
)

# Example after DLP redaction:
save_runtime_event(
    rule_code="DLP-CC-001",
    event_type="RESPONSE_REDACTED",
    ip_address=request.remote_addr,
    http_method=request.method,
```

```
    path=request.path,  
    details="Sensitive response data redacted",  
    action="REDACT",  
    response_status=200,  
)
```

Use the real rule code that triggered the event. Do not log secrets or complete sensitive payloads into details.

MODULE 3 — PRACTICE SHOP + AUTOMATED TESTING

16. Step 14 — Keep Module 3 on port 8001

Do not turn Module 3 into a second database or Supabase project. Keep the existing Flask Practice Shop as the intentionally vulnerable target. Module 2 must remain in front of it.

```
# module3_practice_shop.py
app.run(host="127.0.0.1", port=8001, debug=True)
```

The exact existing Practice Shop routes are: GET /api/products, POST /api/checkout, GET /api/orders/, GET /api/admin/stats, GET /api/search, and GET/POST /api/reviews.

17. Step 15 — Practice scenarios from Supabase

```
# practice_db.py
from supabase_client import supabase

def get_enabled_scenarios():
    return (supabase.table("practice_scenarios")
            .select("""
                .eq("enabled", True)
            .execute().data)
```

18. Step 16 — Run a scenario through Module 2, not directly to Module 3

```
# test flow – conceptual
# Module 2 = http://127.0.0.1:8000
# Module 3 = http://127.0.0.1:8001
#
# Scenario test should send its request to port 8000.
# Module 2 decides BLOCK / ALLOW / REDACT.
# Only allowed requests are forwarded to port 8001.
```

This preserves the TrustIQ architecture. Direct calls to 8001 can be used only for debugging the Practice Shop itself, not for claiming that the Security Guard protected the request.

19. Step 17 — Store practice test results

```
# practice_db.py (continued)
def save_practice_test(
    scenario_id,
    project_id,
    trustiq_rule_code,
    expected_action,
    actual_action,
    expected_blocked,
    actually_blocked,
    status,
    response_status=None,
    message=None,
):
    return (supabase.table("practice_tests").insert({
        "scenario_id": scenario_id,
        "project_id": project_id,
        "trustiq_rule_code": trustiq_rule_code,
        "expected_action": expected_action,
        "actual_action": actual_action,
        "expected_blocked": expected_blocked,
        "actually_blocked": actually_blocked,
        "status": status,
        "response_status": response_status,
        "message": message,
    }).execute().data)
```

20. Step 18 — Test comparison logic

```
def calculate_status(expected_action, actual_action,
                    expected_blocked, actually_blocked):
    if (expected_action == actual_action
        and expected_blocked == actually_blocked):
        return "PASS"
    return "FAIL"
```

Use ERROR when the test itself cannot execute, for example when the proxy or Practice Shop is not running.

21. End-to-end startup order

Always start the three pieces in this order for a full demonstration:

```
TERMINAL 1 – Module 3
python module3_practice_shop.py
# should listen on 8001
```

```
TERMINAL 2 – Module 2
python module2_proxy.py
# should listen on 8000 and target 8001
```

```
TERMINAL 3 – Module 1
python module1_integration.py
# scans the selected repository
```

```
TERMINAL 4 – optional automated practice tests
python run_practice_tests.py
```

22. End-to-end data flow

```
MODULE 1
demo repository
|
+--> secret_rules
+--> sensitive_files
+--> dependency_rules
|
+--> scans
+--> findings
```

```
MODULE 2
client
|
v
:8000 Security Guard
|
+--> security_rules
+--> runtime_events
|
v
:8001 Practice Shop
```

```
PRACTICE TESTER
|
+--> practice_scenarios
|
v
:8000
|
v
Module 2 decision
|
v
practice_tests
```

23. Complete verification checklist

1. Supabase connection succeeds.
2. .env is ignored by Git.
3. Module 1 can read secret_rules, sensitive_files and dependency_rules.
4. Module 1 creates a scan record.
5. Existing Module 1 scanner still detects its original findings.
6. Module 1 writes findings with module = MODULE_1.
7. Module 1 completes the scans row with correct counts.
8. Module 3 starts on port 8001.
9. Module 2 starts on port 8000 and TARGET_URL points to 8001.
10. Module 2 can load enabled security_rules.

11. Module 2 writes runtime_events for blocked/allowed/redacted security events.
12. Module 2 still performs its original rate-limit, auth, IDOR, price, SQLi/XSS and DLP pipeline.
13. Practice scenarios are read from practice_scenarios.
14. Practice requests are sent through port 8000.
15. Practice results are written to practice_tests.
16. PASS/FAIL reflects expected vs actual action and blocked state.
17. Direct 8001 tests are not counted as proxy-protection tests.
18. All demo secrets are fake.
19. No Supabase secret appears in Python source or GitHub.
20. A clean restart of all services reproduces the result.

24. What should remain unchanged

Module 1: its original detection behavior. **Module 2:** its port 8000, target 8001, rate limiting, blocklist, auth/authz/IDOR/price checks, SQLi/XSS request checks, DLP response filtering and dashboard routes. **Module 3:** its intentional demo vulnerabilities and port 8001. Supabase integration should add persistence/configuration, not silently redesign these components.

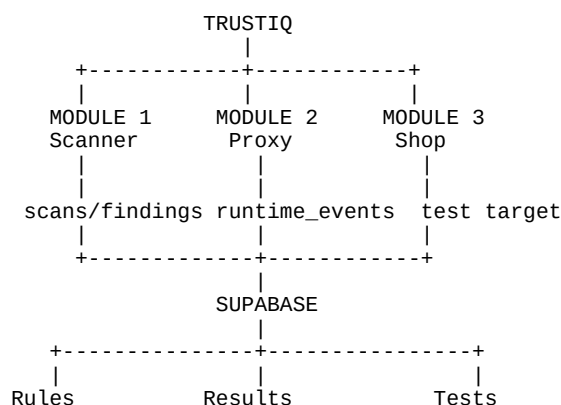
25. Important integration caveats

Your current Module 2 JWT fallback that treats an invalid token as a plain user ID should be corrected during hardening; an invalid JWT must not silently become authenticated identity. Also move its JWT secret to .env as shown above. Do this as a controlled security fix and retest auth/IDOR scenarios afterward.

Your current Module 1 dependency scanner was previously noted to increment errors without always producing dependency findings. Before declaring Module 1 complete, make sure a vulnerable dependency creates a normal findings row with a correct dependency rule code.

The existing Module 2 code currently uses hardcoded DLP patterns and valid prices. The database integration shown here should not remove them immediately. First get persistence working, then migrate individual rule sources one at a time and test after each migration.

26. Final expected state



Do not try to implement everything in one paste. The correct path is: connection test → Module 1 persistence → Module 1 end-to-end → Module 2 persistence → Module 2 end-to-end → Module 3 + scenario testing → full three-module test. This isolates failures and protects the working portions of TrustIQ.